

ĐẠI HỌC HUẾ
TRƯỜNG ĐỊA HỌC KHOA HỌC HUẾ



Phát triển ứng dụng IoT
Đề tài: Điều khiển quạt dựa trên nhiệt độ với ESP32

Giảng viên giảng dạy: Võ Việt Dũng

Sinh viên thực hiện: Nguyễn Đình Vĩnh Phú

Mã Sinh Viên: 22T1020322

Khóa: K46

DANH MỤC CÁC TỪ VIẾT TẮT

Từ viết tắt	Tiếng Anh	Tiếng Việt
IoT	Internet of Things	Internet vạn vật
GPIO	General-Purpose Input/Output	Chân vào/ra đa dụng
I2C	Inter-Integrated Circuit	Giao thức giao tiếp vi mạch
OLED	Organic Light Emitting Diode	Đi-ốt phát sáng hữu cơ
SoC	System on a Chip	Hệ thống trên một vi mạch
PWM	Pulse Width Modulation	Điều chế độ rộng xung
NaN	Not a Number	Không phải số (giá trị lỗi)
AC	Alternating Current	Dòng điện xoay chiều

MỤC LỤC

PHẦN MỞ ĐẦU.....	3
1. Lý do chọn đề tài:.....	3
2. Mục tiêu nghiên cứu.....	3
3. Đối tượng và phạm vi nghiên cứu.....	4
PHẦN NỘI DUNG.....	4
CHƯƠNG 1: CƠ SỞ LÝ THUYẾT.....	4
1.1. Vi điều khiển ESP32:.....	4
<i>Thông số kỹ thuật chính của ESP32:</i>	5
1.2. Cảm biến nhiệt độ và độ ẩm DHT22:.....	5
1.3. Module Relay và Nguyên lý cách ly quang (Opto-isolation):.....	6
1.4. Màn hình OLED SSD1306:.....	7
1.5. Nền tảng IoT đám mây ThingSpeak:.....	7
CHƯƠNG 2: THIẾT KẾ HỆ THỐNG VÀ THUẬT TOÁN ĐIỀU KHIỂN.....	8
2.1. Thiết kế phần cứng:.....	8
<i>Bảng kết nối chi tiết:</i>	8
2.2. Thuật toán điều khiển trễ (Hysteresis Algorithm):.....	9
2.3. Xây dựng logic phần mềm:.....	10

<i>Cơ chế Fail-safe:</i>	10
2.4. Tích hợp nền tảng IoT ThingSpeak:.....	10
CHƯƠNG 3: KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ.....	10
3.1. Môi trường kiểm thử:.....	10
3.2. Kết quả kiểm thử:.....	12
3.3. Đánh giá tổng thể:.....	13
<i>Ưu điểm:</i>	13
<i>Nhược điểm và hạn chế:</i>	13
PHẦN KẾT LUẬN, ĐÁNH GIÁ VÀ KIẾN NGHỊ.....	13
1. Kết luận:.....	13
2. Kiến nghị và Hướng phát triển:.....	13
TÀI LIỆU THAM KHẢO.....	14
PHỤ LỤC.....	14
Phụ lục 1: Sơ đồ nối dây (Wokwi – diagram.json).....	14
Phụ lục 2: Mã nguồn main.cpp.....	14

PHẦN MỞ ĐẦU

1. Lý do chọn đề tài:

Trong bối cảnh Internet of Things (IoT) đang phát triển mạnh mẽ, việc tự động hóa các thiết bị gia dụng không chỉ mang lại sự tiện lợi mà còn đóng vai trò cốt lõi trong việc tối ưu hóa năng lượng. Việc sử dụng quạt máy với một trạng thái cố định thường dẫn đến tình trạng lãng phí điện năng khi trời lạnh, hoặc gây ảnh hưởng không tốt đến sức khỏe lúc ngủ say.

Xuất phát từ thực tiễn đó, nhóm tiến hành nghiên cứu đề tài "Điều khiển quạt dựa trên nhiệt độ với ESP32" nhằm tự động đóng/ngắt thiết bị làm mát dựa trên dữ liệu môi trường thực tế, góp phần hướng đến xây dựng ngôi nhà thông minh tiết kiệm năng lượng.

Tích hợp nền tảng đám mây (Cloud) để đẩy dữ liệu lên Internet, nâng cấp hệ thống thành một trạm giám sát IoT hoàn chỉnh có khả năng trực quan hóa dữ liệu từ xa.

2. Mục tiêu nghiên cứu

- Nghiên cứu vi điều khiển ESP32 và các chuẩn giao tiếp ngoại vi.
- Thiết kế mạch phần cứng tích hợp cảm biến DHT22, OLED và Relay.
- Xây dựng thuật toán điều khiển có dải trễ (Hysteresis) để bảo vệ thiết bị.
- Lập trình cơ chế Fail-safe đảm bảo an toàn khi cảm biến gặp sự cố

3. Đối tượng và phạm vi nghiên cứu

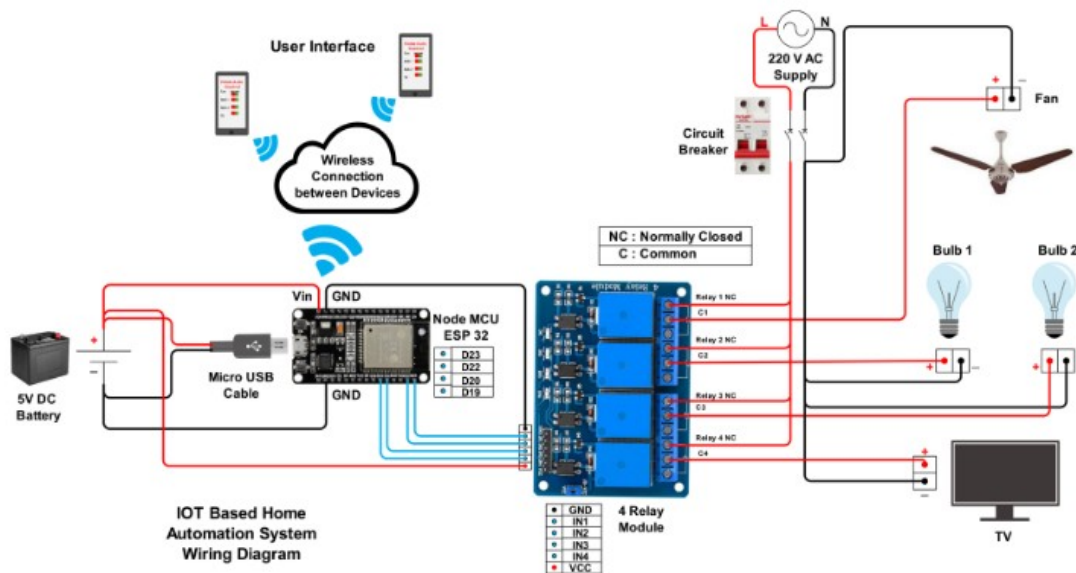
- **Đối tượng:** Vi điều khiển ESP32, Cảm biến nhiệt độ và độ ẩm DHT22, Module Relay 5V, Màn hình OLED SSD1306 và Nền tảng đám mây ThingSpeak.
- **Phạm vi:** Thiết kế mạch phần cứng, lập trình logic điều khiển tự động Hysteresis tại biên (Edge) và truyền tải dữ liệu môi trường qua Wi-Fi lên máy chủ ThingSpeak để trực quan hóa.

PHẦN NỘI DUNG

CHƯƠNG 1: CƠ SỞ LÝ THUYẾT

1.1. Vi điều khiển ESP32:

ESP32 là hệ thống trên một vi mạch (SoC) được phát triển bởi Espressif Systems. Với xung nhịp lên đến 240MHz và bộ nhớ SRAM 520KB, ESP32 sở hữu sức mạnh xử lý vượt trội so với các vi điều khiển truyền thống, đáp ứng tốt nhu cầu thu thập dữ liệu liên tục từ cảm biến.



Thông số kỹ thuật chính của ESP32:

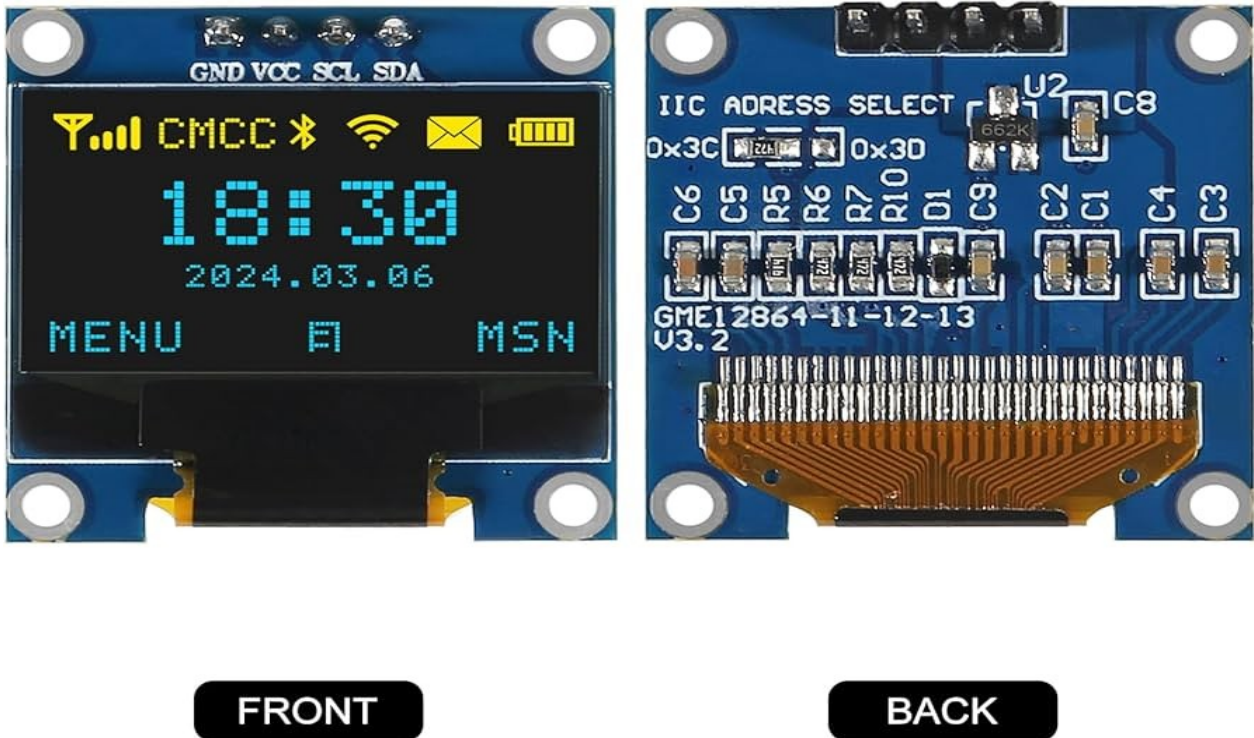
Thông số	Giá trị
CPU	Dual-core Xtensa LX6, tốc độ 240 MHz
SRAM	520 KB
Flash (ngoài)	4 MB (tùy module)
GPIO	34 chân (một số chỉ đọc: 34–39)
ADC	12-bit, 18 kênh
Giao thức	I2C, SPI, UART, I2S, CAN
Kết nối không dây	Wi-Fi 2.4 GHz, Bluetooth 4.2
Điện áp hoạt động	3.3 V (chân GPIO); cấp nguồn 5 V qua USB

1.2. Cảm biến nhiệt độ và độ ẩm DHT22:

DHT22 là cảm biến kỹ thuật số sử dụng chuẩn giao tiếp 1-Wire. Dải đo nhiệt độ của DHT22 trải dài từ -40°C đến 80°C với sai số $\pm 0.5^{\circ}\text{C}$ và dải đo độ ẩm từ 0% đến 100% RH với sai số $\pm 2\%$, hoàn toàn phù hợp để giám sát nhiệt độ phòng sinh hoạt thời gian thật.

1.4. Màn hình OLED SSD1306:

SSD1306 là module màn hình OLED đơn sắc kích thước 0.96 inch, độ phân giải 128×64 pixel, giao tiếp qua chuẩn I2C với địa chỉ mặc định 0x3C. Màn hình không cần đèn nền, tiêu thụ điện cực thấp (khoảng 20 mA) và hiển thị rõ ràng ngay cả trong môi trường ánh sáng yếu – lý tưởng cho các thiết bị IoT nhúng.



FRONT

BACK

Hình ảnh: Ảnh thực tế Module OLED SSD1306

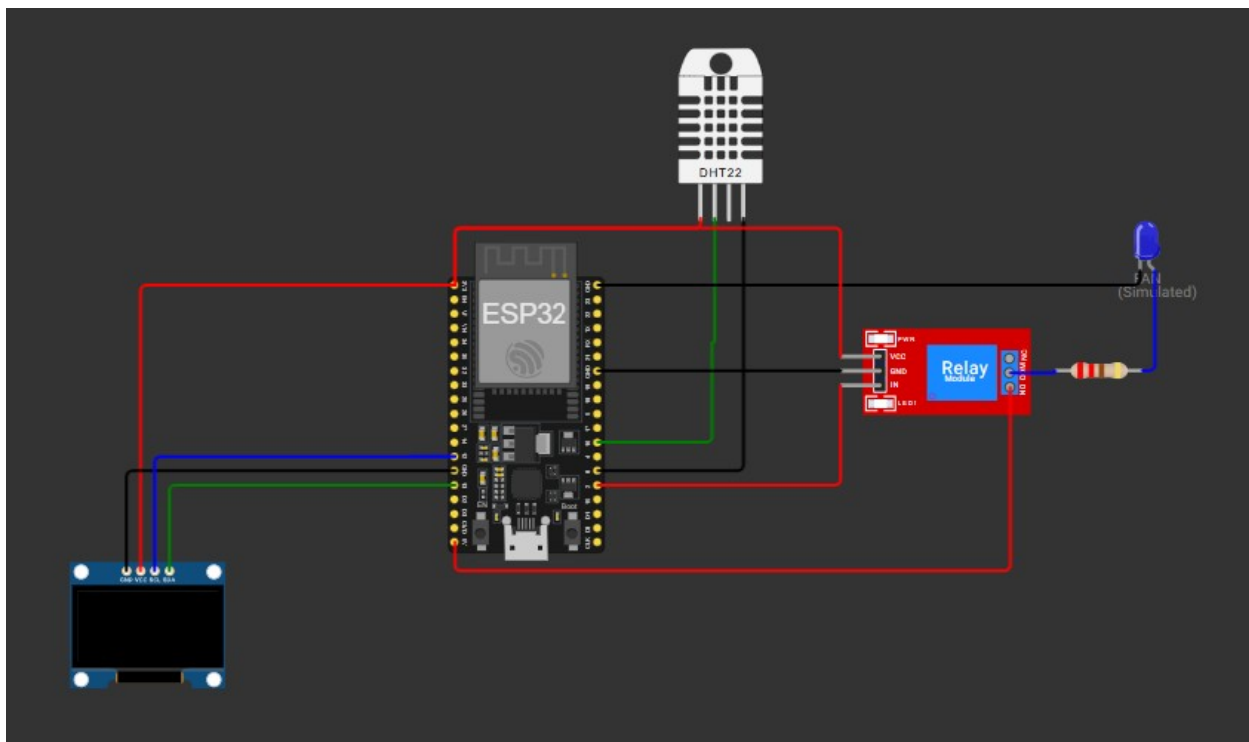
1.5. Nền tảng IoT đám mây ThingSpeak:

ThingSpeak là một dịch vụ nền tảng phân tích IoT (Internet of Things) mã nguồn mở, cho phép người dùng tổng hợp, trực quan hóa và phân tích các luồng dữ liệu trực tiếp trên đám mây. Được phát triển bởi MathWorks, ThingSpeak cung cấp các API RESTful để thiết bị phần cứng (như ESP32) có thể dễ dàng gửi dữ liệu lên máy chủ thông qua giao thức HTTP (phương thức GET/POST). Trong dự án nhà thông minh này, ThingSpeak đóng vai trò như một cơ sở dữ liệu chuỗi thời gian (Time-series Database), tự động lưu trữ các giá trị đo lường từ cảm biến và biểu diễn chúng dưới dạng biểu đồ đồ họa (Dashboard) theo thời gian thực. Điều này giúp hệ thống mở rộng từ một mạch điều khiển offline thành một trạm giám sát IoT hoàn chỉnh có khả năng truy cập từ xa.

CHƯƠNG 2: THIẾT KẾ HỆ THỐNG VÀ THUẬT TOÁN ĐIỀU KHIỂN

2.1. Thiết kế phần cứng:

Hệ thống được thiết kế theo kiến trúc tập trung, với ESP32 DevKit C V4 đóng vai trò bộ xử lý trung tâm. DHT22 kết nối qua GPIO 16 (1-Wire) làm đầu vào đọc dữ liệu môi trường. Màn hình OLED SSD1306 kết nối qua I2C (SDA: GPIO 13, SCL: GPIO 12) để hiển thị thông tin thời gian thực. Module Relay kết nối tín hiệu IN qua GPIO 2 để điều khiển tải AC.



Hình ảnh: Sơ đồ nối dây đầy đủ trên Wokwi

Bảng kết nối chi tiết:

Linh kiện	Chân linh kiện	Chân ESP32	Màu dây
DHT22	SDA (Data)	GPIO 16	Xanh lá
DHT22	VCC	3.3 V	Đỏ
DHT22	GND	GPIO 0 (GND)	Đen

Linh kiện	Chân linh kiện	Chân ESP32	Màu dây
OLED SSD1306	SDA	GPIO 13	Xanh lá
OLED SSD1306	SCL	GPIO 12	Xanh dương
OLED SSD1306	VCC	3.3 V	Đỏ
OLED SSD1306	GND	GND	Đen
Module Relay	IN	GPIO 2	Đỏ
Module Relay	VCC	5 V	Đỏ
Module Relay	GND	GND	Đen
Module Relay	NO	5 V	Đỏ
LED xanh (giả lập quạt)	Anode	Relay NO $\rightarrow$ R 220 Ω	Xanh
LED xanh (giả lập quạt)	Cathode	GND	Đen

2.2. Thuật toán điều khiển trễ (Hysteresis Algorithm):

Để tránh hiện tượng chattering – relay đóng/ngắt liên tục khi nhiệt độ dao động sát ngưỡng – thuật toán Hysteresis (điều khiển có dải trễ) được áp dụng. Hệ thống định nghĩa hai ngưỡng tách biệt thay vì một ngưỡng duy nhất:

- TEMP_ON = 30.0°C: Ngưỡng nhiệt độ CAO – kích hoạt bật quạt.
- TEMP_OFF = 28.5°C: Ngưỡng nhiệt độ THẤP – vô hiệu hóa, tắt quạt.
- Dải trễ = 1.5°C: Khoảng đệm ngăn relay rung.

Khi nhiệt độ nằm trong dải $28.5^{\circ}\text{C} - 30.0^{\circ}\text{C}$, hệ thống duy trì nguyên trạng thái hiện tại (đang bật thì bật tiếp, đang tắt thì tắt tiếp). Điều này giúp bảo vệ tiếp điểm relay và kéo dài tuổi thọ thiết bị đáng kể.

2.3. Xây dựng logic phần mềm:

Non-blocking Timer với millis():

Thay vì dùng hàm `delay()` gây treo CPU, chương trình sử dụng hàm `millis()` để đo thời gian trôi qua kể từ lần đọc cuối cùng. CPU có thể xử lý các tác vụ khác (nhận nút bấm, kết nối Wi-Fi...) trong khoảng thời gian chờ 2 giây giữa các lần đọc cảm biến.

Cơ chế Fail-safe:

Nếu DHT22 mất kết nối do đứt dây hoặc nhiễu, hàm `readTemperature()` trả về NaN (Not a Number). Chương trình kiểm tra điều kiện `isnan()` và ngay lập tức ngắt Relay về LOW, hiển thị cảnh báo "SENSOR ERROR" trên OLED, sau đó thoát khỏi vòng lặp hiện tại qua lệnh `return` – tránh mọi thao tác điều khiển tiếp theo.

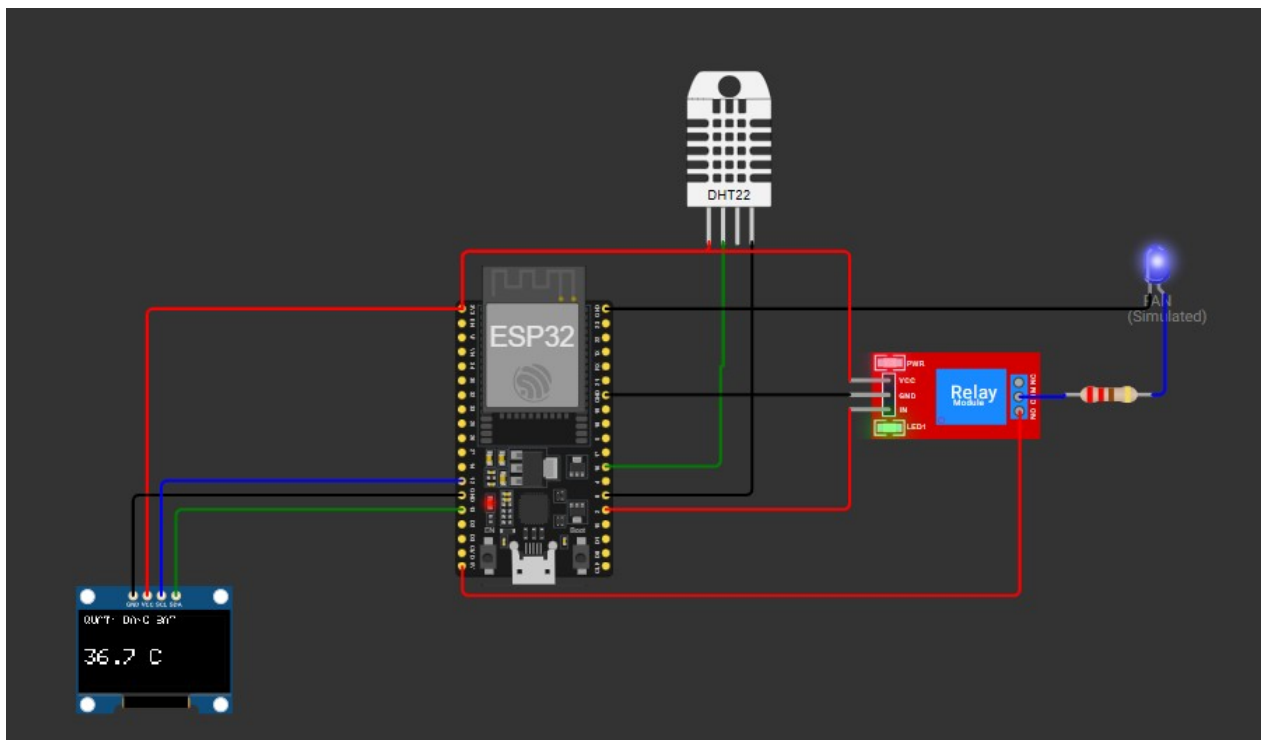
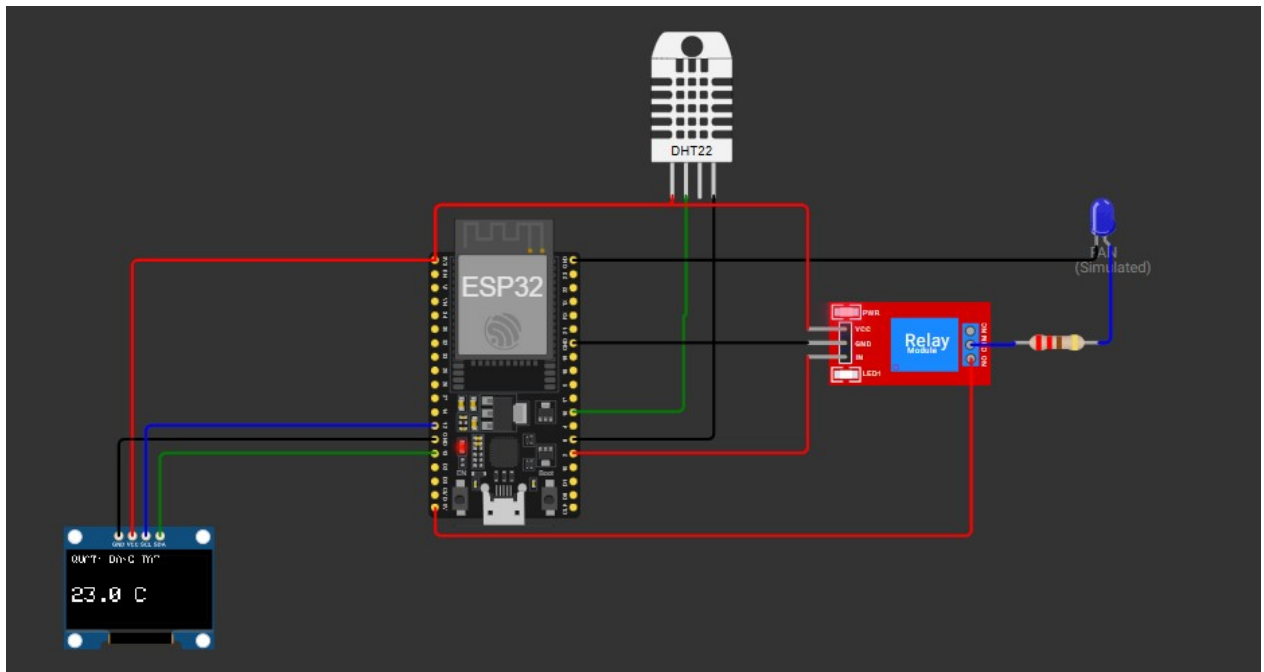
2.4. Tích hợp nền tảng IoT ThingSpeak:

Để đáp ứng đầy đủ tiêu chí của một hệ thống Internet of Things (IoT) thực thụ, ngoài việc xử lý tự động hóa tại biên (Edge Computing) bằng Hysteresis, hệ thống còn được tích hợp khả năng giám sát từ xa thông qua nền tảng đám mây ThingSpeak. ESP32 sử dụng module Wi-Fi tích hợp sẵn để kết nối Internet. Định kỳ mỗi 15 giây (khoảng thời gian tối ưu cho phép bởi ThingSpeak API), vi điều khiển sẽ đóng gói dữ liệu bao gồm Nhiệt độ, Độ ẩm và Trạng thái Relay hiện tại để gửi lên máy chủ thông qua giao thức HTTP GET. Dữ liệu này được ThingSpeak trực quan hóa thành các biểu đồ theo thời gian thực (Time-series charts), giúp người quản trị có thể theo dõi tình trạng môi trường và lịch sử hoạt động của quạt từ bất kỳ đâu có kết nối Internet mà không làm gián đoạn tiến trình điều khiển mạch ngoại vi.

CHƯƠNG 3: KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ

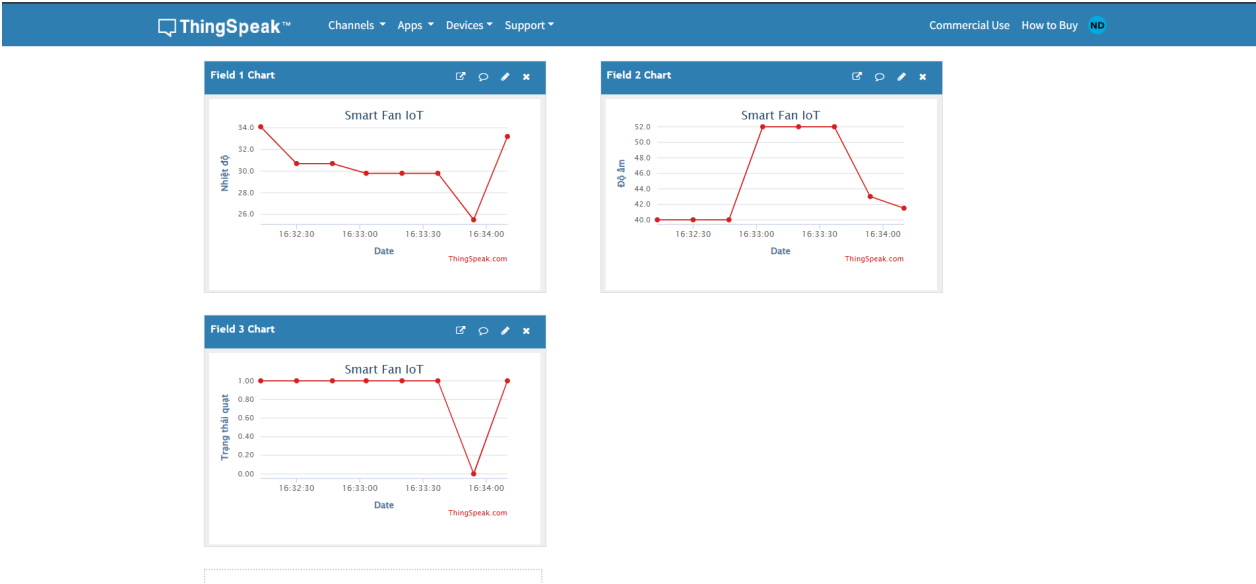
3.1. Môi trường kiểm thử:

Hệ thống được mô phỏng và kiểm thử trên nền tảng Wokwi – một trình mô phỏng phần cứng trực tuyến hỗ trợ đầy đủ ESP32, DHT22, SSD1306 và Module Relay. Việc điều chỉnh nhiệt độ ảo trên Wokwi cho phép kiểm tra chính xác các ngưỡng điều khiển mà không cần phần cứng thực.



3.2. Kết quả kiểm thử:

Kịch bản kiểm thử	Nhiệt độ đầu vào	Trạng thái relay	Hiển thị OLED	Kết quả
Nhiệt độ thấp	< 28.5°C	OFF (LOW)	DUNG (OFF)	✓ Đạt
Nhiệt độ cao	>= 30.0°C	ON (HIGH)	DANG CHAY (ON)	✓ Đạt
Dải trễ (bật → giữ)	29.5°C	ON (giữ)	DANG CHAY (ON)	✓ Đạt
Dải trễ (tắt → giữ)	29.0°C	OFF (giữ)	DUNG (OFF)	✓ Đạt
Mất cảm biến (NaN)	NaN	OFF (LOW)	SENSOR ERROR!	✓ Đạt
Khởi động lạnh	Bất kỳ	OFF (LOW)	SMART FAN SYSTEM	✓ Đạt



3.3. Đánh giá tổng thể:

Ưu điểm:

- Thuật toán Hysteresis hoạt động chính xác, loại bỏ hoàn toàn hiện tượng chattering.
- Cơ chế Fail-safe đảm bảo an toàn điện khi cảm biến gặp sự cố.
- Non-blocking timer giữ CPU luôn sẵn sàng, không bị treo khi chờ cảm biến.
- Giao diện OLED hiển thị đầy đủ thông tin: nhiệt độ, độ ẩm, trạng thái quạt.
- Mạch cách ly quang bảo vệ ESP32 khỏi xung điện áp ngược từ tải AC.

Nhược điểm và hạn chế:

- Chỉ điều khiển ON/OFF, chưa thay đổi được tốc độ quạt vô cấp.
- Chưa tích hợp Wi-Fi để giám sát và điều khiển từ xa qua ứng dụng di động.
- Ngưỡng nhiệt độ cố định trong mã nguồn, chưa có giao diện cài đặt linh hoạt.

PHẦN KẾT LUẬN, ĐÁNH GIÁ VÀ KIẾN NGHỊ

1. Kết luận:

Đề tài đã hoàn thành việc thiết kế mạch và lập trình thành công hệ thống điều khiển quạt tự động dựa trên nhiệt độ môi trường sử dụng vi điều khiển ESP32. Thuật toán Hysteresis với dải trễ 1.5°C và cơ chế Fail-safe đã được tích hợp chuẩn xác, giúp hệ thống vận hành ổn định, an toàn và bền bỉ.

Kết quả kiểm thử trên Wokwi xác nhận tất cả các kịch bản hoạt động đúng như thiết kế, bao gồm cả trường hợp ngoại lệ khi cảm biến bị lỗi. Đây là nền tảng vững chắc để phát triển thành hệ thống nhà thông minh hoàn chỉnh trong tương lai.

2. Kiến nghị và Hướng phát triển:

Thay thế Module Relay bằng module Dimmer (Triac/BTA16) để điều khiển tốc độ quạt vô cấp bằng PWM, tiết kiệm điện năng hơn nữa.

- Bổ sung nút bấm vật lý để người dùng có thể chỉnh ngưỡng TEMP_ON và TEMP_OFF trực tiếp mà không cần nạp lại firmware.
- Lưu lịch sử nhiệt độ vào bộ nhớ SPIFFS của ESP32 để phân tích xu hướng nhiệt độ theo thời gian.

TÀI LIỆU THAM KHẢO

- [1]. Adafruit Industries (2022), DHT22 Temperature-Humidity Sensor Datasheet, truy cập tại: <https://www.adafruit.com/product/385>.
- [2]. Espressif Systems (2023), ESP32 Technical Reference Manual v5.0, truy cập tại: <https://docs.espressif.com>.
- [3]. Solomon Systech (2008), SSD1306 OLED Driver IC Datasheet, Solomon Systech Limited.

PHỤ LỤC

Phụ lục 1: Sơ đồ nối dây (Wokwi – diagram.json)

[HÌNH ẢNH: Ảnh chụp toàn bộ sơ đồ kết nối trên Wokwi (zoom out để thấy toàn mạch)]

Phụ lục 2: Mã nguồn main.cpp

```
#include <Arduino.h>

#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <DHT.h>
#include <WiFi.h>
#include <HttpClient.h>

// --- CẤU HÌNH WIFI & THINGSPEAK ---
const char* ssid = "Wokwi-GUEST"; // Wi-Fi mặc định của Wokwi (Không cần pass)
const char* password = "";
String serverName = "http://api.thingspeak.com/update";
String apiKey = "3T7EL7AJ0HGBNSD1";

// --- CẤU HÌNH PHẦN CỨNG ---
#define PIN_DHT 16
#define DHT_TYPE DHT22
#define PIN_RELAY 2
#define PIN_OLED_SDA 13
```

```

#define PIN_OLED_SCL    12
#define SCREEN_WIDTH    128
#define SCREEN_HEIGHT   64

// --- CẤU HÌNH LOGIC (HYSTERESIS) ---
#define TEMP_ON         30.0
#define TEMP_OFF        28.5

DHT dht(PIN_DHT, DHT_TYPE);
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

unsigned long lastReadTime = 0;
unsigned long lastThingSpeakTime = 0;
bool isFanOn = false;

void setup() {
    Serial.begin(115200);

    pinMode(PIN_RELAY, OUTPUT);
    digitalWrite(PIN_RELAY, LOW);

    dht.begin();

    Wire.begin(PIN_OLED_SDA, PIN_OLED_SCL);
    if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
        Serial.println(F("OLED ERROR!"));
        while (true) delay(100);
    }

    // --- KẾT NỐI WIFI ---
    display.clearDisplay();
    display.setTextColor(WHITE);
    display.setTextSize(1);
    display.setCursor(0, 20);
    display.println("Connecting Wi-Fi...");
    display.display();

    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }
    Serial.println("\nWiFi Connected!");

    display.clearDisplay();

```



```

display.setCursor(0, 20);
display.println("Wi-Fi Connected!");
display.display();
delay(1000);
}

void loop() {
  // 1. LOGIC ĐIỀU KHIỂN & OLED (Chạy mỗi 2 giây)
  if (millis() - lastReadTime >= 2000) {
    lastReadTime = millis();
    float t = dht.readTemperature();
    float h = dht.readHumidity();

    if (isnan(t) || isnan(h)) {
      digitalWrite(PIN_RELAY, LOW);
      isFanOn = false;
      display.clearDisplay();
      display.setCursor(10, 20);
      display.setTextSize(2);
      display.println("SENSOR ERR");
      display.display();
      return;
    }

    if (t >= TEMP_ON && !isFanOn) {
      digitalWrite(PIN_RELAY, HIGH);
      isFanOn = true;
    } else if (t <= TEMP_OFF && isFanOn) {
      digitalWrite(PIN_RELAY, LOW);
      isFanOn = false;
    }

    // Hiện OLED
    display.clearDisplay();
    display.drawRect(0, 0, 128, 64, WHITE);
    display.setTextSize(1);
    display.setCursor(5, 5);
    display.print("QUAT: ");
    display.println(isFanOn ? "ON" : "OFF");
    display.drawLine(0, 18, 128, 18, WHITE);
    display.setTextSize(2);
    display.setCursor(5, 25);
    display.print(t, 1);
    display.print("C");
    display.setTextSize(1);

```

```

display.setCursor(85, 25);
display.print("AM D0");
display.setCursor(85, 40);
display.print(h, 0);
display.print("%");
display.display();

// 2. LOGIC ĐẨY DATA LÊN THINGSPEAK (Chạy mỗi 15 giây)
if (WiFi.status() == WL_CONNECTED && millis() - lastThingSpeakTime >= 15000)
{
    lastThingSpeakTime = millis();
    HTTPClient http;
    String serverPath = serverName + "?api_key=" + apiKey + "&field1=" +
String(t) + "&field2=" + String(h) + "&field3=" + String(isFanOn);

    http.begin(serverPath.c_str());
    int httpResponseCode = http.GET();

    if (httpResponseCode > 0) {
        Serial.print("ThingSpeak Update Success: ");
        Serial.println(httpResponseCode);
    } else {
        Serial.print("ThingSpeak Error: ");
        Serial.println(httpResponseCode);
    }
    http.end();
}
}
}

```

— Hết báo cáo —